

Lab 12 - Cloud, Local and Hybrid Production Architecture Evaluation

Course: Create Short Video Film using AI (TGS-2020505925) | Version v12.0 | 27 September 2026 Topic 05: Cloud and Edge AI Workflows for Short Film Production Outcome: ELO6 | In-class time: 60 minutes of scheduled practical time

Competency	Statement
Ability A7	Design the architecture of applied vision systems
Ability A8	Design, develop and evaluate edge-based and cloud-based systems
Knowledge K11	Vision system architecture
Knowledge K12	Vision communication protocols
Knowledge K13	Real-world design constraints and solution options

Scenario

Harbourline wants The Last Ferry and, if it works, a twice-weekly series. You design the production system: an all-in-one drama agent, a multi-tool cloud pipeline, a hybrid (cloud generation, local post-production) or a local open video model. You measure the local stages on your own laptop, model generation cost and time for each architecture, check whether the open model fits your GPU, apply the gates, and defend a recommendation.

What you will produce

A four-layer architecture diagram for your chosen design, an architecture matrix (cost, cost per finished minute, generation time, upload time, end-to-end hours, VRAM fit, gates), the protocol sequence for your design, and two what-if results.

Tools: Python 3 | opencv-python | NumPy | a pen or diagram tool | no paid service required

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: opencv-python>=4.10, numpy.

API currency. Everything taught here runs on the base opencv-python wheel plus NumPy - no contrib modules and no downloaded model weights. SSIM is implemented in NumPy because cv2.quality is contrib-only. Generative steps are optional: every lab supplies SIMULATED reference media so it can be completed offline, and asks you to record the model ID and date if you generate your own.

Workflow

1. Draw the four-layer architecture
2. Measure the local stages on this machine
3. Model cost, time and upload per architecture
4. Apply gates, then rank
5. Run what-ifs and defend the choice

Files in this folder

Path	What it is
reference/bench-clip.mp4	4-second 960x540 clip used to time the local stages (SIMULATED clip)
reference/element-pack/	The saved references (character sheet, two location plates) that a cloud architecture must upload
data/production-plan.json	Seven shots, seconds and takes for The Last Ferry
data/architectures.json	Four architectures: ILLUSTRATIVE rates, speeds, regions, quality, control and protocols
data/constraints.json	Budget, deadline, uplink, regions, likeness, quality, control and GPU memory
production_arch.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom illustration of *The Last Ferry*, rendered deterministically with OpenCV and labelled on the frame, or an AI-generated reference copied unchanged with a PROVENANCE.md beside it. Nothing in this folder may be relabelled: a simulated stand-in is never presented as model output or as a photograph.

Steps

1. Draw the four layers for The Last Ferry in out/architecture.png or on paper: ASSETS (script, element pack, voice) → LOCAL WORKSTATION (QA analytics, grade, captions, mix, export) → CLOUD GENERATION (image, video, voice, music services) → DELIVERY (platforms, archive). Mark what crosses each boundary.
2. Read data/architectures.json. Note that every rate is ILLUSTRATIVE and must be replaced with the current price list before quoting a client.
3. Read data/constraints.json: S\$400 budget, 24-hour deadline, 20 Mbps uplink, a brand-hero quality tier of 4 and shot-level control required.
4. Run the evaluation.

```
python3 production_arch.py --plan data/production-plan.json --archs data/architectures.json
--constraints data/constraints.json --clip reference/bench-clip.mp4 --elements reference/
element-pack --out out
```

5. Section 1 is MEASURED on your machine: seconds of work per second of footage for decode, QA analytics, grading and encoding. Compare with a classmate's laptop.

6. Section 2 is MODELLED: 98 generated seconds for a 48-second film because of extra takes. Check one architecture's cost by hand: generated seconds × credits per second × S\$ per credit + fixed cost.
7. Read the VRAM line for the local model: a 14-billion-parameter model needs about 36 GB at FP16, 18 GB at INT8 and 9 GB at INT4 (weights × bytes × 1.3 for activations). Only INT4 fits a 16 GB GPU.
8. Read the gates. A fails quality and shot control; D fails quality. B and C pass; C is cheaper. Write why a gate is applied before ranking.
9. Describe the protocol sequence of your recommended architecture in out/protocol-sequence.md: signed-URL upload of the element pack, JSON POST per shot, the operation ID, polling or webhook, and the signed-URL download.
10. What-if 1: set likeness_data_must_stay_local to true (as it would be for a film using a real employee's face) and rerun. Record which architectures survive.
11. What-if 2: restore it, then lower min_quality_tier to 2 (a social draft). Rerun and record how the recommendation and cost per finished minute change.
12. Write out/recommendation.md: the architecture, the numbers that justify it, the gate that would overturn it, and the disclosure and provenance steps (C2PA / watermark, platform AI label) your pipeline performs before delivery.

Verify

The matrix lists four architectures; A and D fail gates with named reasons; C is recommended; both what-ifs are recorded with their new outcome.

Expected outputs

- out/architecture-matrix.csv and out/architecture-report.json
- out/bench-encode.mp4 (the timed local encode)
- out/architecture.png (or a photo of your drawing)
- out/protocol-sequence.md and out/recommendation.md

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- [] Four-layer architecture diagram with the boundary payloads marked
- [] MEASURED local timings from your machine
- [] Hand check of one MODELLED cost and the VRAM calculation
- [] Protocol sequence for the recommended architecture
- [] Two what-if results and the final recommendation with its overturning gate

Troubleshooting

Symptom	What to do
Local timings are much faster or slower than a classmate's	Expected: they are measured on each machine. Report yours with the hardware.
Every architecture fails	Change the plan (fewer takes, lower draft resolution, longer deadline) - never relax a gate to manufacture a winner.
cannot read the bench clip	Run from inside the lab folder; reinstall opencv-python if VideoCapture fails.
Our real prices differ	Good - put the current published rates in architectures.json and rerun.

References used in this lab

- [S12] Sanky, 'How I Made an AI Short Drama in under 10 Minutes with ZooClaw + Seedance2', YouTube, 2 Jun 2026, youtube.com/watch?v=q8z5iK5Pm_s - *Chapters and description, 7:47*.

An all-in-one drama agent: upload one photo as the protagonist, describe the story, and the agent produces character-consistent shots, voiceover, music, storyboard and video; the prompt formula (premise, main character, scene style, numbered story beats, 'include voiceover, music and subtitles'); trade-off against node-based workflows.

- [S13] M. Tschochohei and F. Schenker, 'Multimodal Generative AI in the Enterprise: From Pixels to Profit', Packt, June 2026, ISBN 978-1-80611-167-1 - *Ch.5 pp.56-75; Ch.7 pp.106-119; Ch.8 pp.127-143*.

VAE, GAN and diffusion mechanisms and failure modes; prompt structure and subject lock; binary masks (white = editable), inpainting, outpainting and mask-free editing; spatial versus temporal coherence, 3-D U-Nets and diffusion transformers over spatiotemporal patches; long-running operation polling; responsible-AI risks (voice cloning, deepfakes, likeness) and transparency (C2PA Content Credentials, invisible watermarking, model cards).

- [S14] A. Mewada, M. A. Ansari, S. Ahmad and N. Singh (eds), 'AI-Generated Image and Video Synthesis', IEEE Press / Wiley, 2026, ISBN 9781394403110 - *Ch.1 pp.1-11; Ch.3 pp.35-44; Ch.7 pp.94-100*.

Frame-based versus temporal video generation and the temporal-consistency challenge; the sensors-to-synthesis layered architecture and split computing between edge and cloud; evaluation (FID, KID, LPIPS, CLIP faithfulness, FVD plus optical-flow agreement); memory = parameters x precision, quantisation to b bits scales by b/32, diffusion latency proportional to S steps (S x F for video).

- [S16] Google AI for Developers - Veo video generation documentation - [*https://ai.google.dev/gemini-api/docs/veo](https://ai.google.dev/gemini-api/docs/veo) (verified 6 Sep 2026)*.

Veo 3.1 model family with native audio; 4, 6 or 8 second clips; 16:9 or 9:16; 720p, 1080p or 4k; up to three reference images on supported models; image-to-video; first and last frame control; clip extension; asynchronous long-running operation that the client polls.

- [S17] OpenCV 4.13 Python package and FFmpeg, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92); ffmpeg/ffprobe optional*.

Every lab command runs on the base opencv-python wheel plus NumPy: VideoCapture / VideoWriter, cvtColor, calcHist/compareHist, Canny, ORB_create + BFMatcher, HuMoments, inpaint, CLAHE, filter2D, fastNIMeansDenoisingColored, Haar cascades from cv2.data.harcascades, HOGDescriptor_getDefaultPeopleDetector, CamShift and calcOpticalFlowFarneback. SSIM is implemented in NumPy because cv2.quality is contrib-only. FFmpeg is used only when present, to mux audio into the final export.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v12.0.